

Formalizing the Jordan-Chevalley Decomposition with a Lean Proving Agent

Name: Ge Tiancheng

Student ID: EEE2509015

Course: AI for Mathematics

Shareable proof link:

<https://tiancheng2016-cell.github.io/JordanChevalleyFormalization/>

Source repository:

<https://github.com/tiancheng2016-cell/JordanChevalleyFormalization>

1. Theorem

I formalized the Jordan-Chevalley decomposition for linear operators. Informally, if V is a finite-dimensional vector space over an algebraically closed field K , then every endomorphism $T : V \rightarrow V$ has a unique decomposition $T = S + N$, where S is semisimple, N is nilpotent, S and N commute, and both parts are polynomial expressions in T .

The final Lean theorem statement was:

```
theorem jordan_chevalley_decomposition
  {K V : Type*} [Field K] [IsAlgClosed K] [AddCommGroup V] [Module K V]
  [FiniteDimensional K V] (T : Module.End K V) :
  ∃ S N : Module.End K V,
    T = S + N ∧
    S.IsSemisimple ∧
    IsNilpotent N ∧
    Commute S N ∧
    S ∈ Algebra.adjoin K ({T} : Set (Module.End K V)) ∧
    N ∈ Algebra.adjoin K ({T} : Set (Module.End K V)) ∧
    ∀ S' N' : Module.End K V,
      T = S' + N' →
      S'.IsSemisimple →
      IsNilpotent N' →
      Commute S' N' →
      S' = S ∧ N' = N := by
```

The clauses $S \in K[T]$ and $N \in K[T]$, implemented by the two `Algebra.adjoin` membership conditions in the displayed Lean statement, express that both parts are polynomial expressions in T . The theorem statement was preserved during the final accepted run and was not weakened.

2. Agent System and Workflow

Codex was the agentic coding assistant and backend. Numina-Lean-Agent was the Lean proving-agent harness. Because Numina normally calls Claude Code, I added a local wrapper adapter that translated Numina’s Claude-shaped subprocess calls into `codex.cmd` `exec` calls and returned the JSON result shape expected by Numina.

I first attempted Archon. It installed and initialized, but its proof loop could not run because Claude Code authentication was unavailable. I investigated Archon-to-Codex re-pointing, but did not patch it: this Archon version was tightly coupled to Claude-specific flags, stream-json parsing, session/resume behavior, authentication checks, and `.claude/` tooling. I therefore switched to the Numina/Codex wrapper route.

3. Prompts Used

The exact full natural-language prompts and configuration files are included in the attached `agent_config_and_prompts.zip`. The PDF report itself includes selected exact prompt excerpts used during the run. Each block below is quoted from the actual prompt log or prompt file; long prompts are shortened only for the page limit, and the full versions are attached.

Prompt A: project preparation. Purpose: create the project structure before Lean, Archon, or proof work.

Please do only project-folder preparation. Do not set up Lean yet.

Tasks:

1. Confirm that these folders exist:
 - inputs
 - logs
 - agent_artifacts/archon_task

Prompt B: Numina/Codex wrapper dry run. Purpose: test the wrapper on a toy Lean file only.

This is a toy adapter test for Numina-Lean-Agent using a Codex wrapper.

Do not edit ``Formalization.lean``.
Only edit ``NuminaAdapterToy.lean``.
Replace the toy ``sorry`` with a valid proof.

Prompt C: real theorem Round 1. Purpose: start the first controlled real theorem proof run.

This is the Jordan-Chevalley decomposition formalization task.

You are running through Numina-Lean-Agent via a Codex wrapper for one controlled round only.

Work only on ``Formalization.lean`` in the current Lean project directory.
Do not edit ``Formalization.lean.bak_step8ea``.

Prompt D: strict repair Round 2. Purpose: steer away from the Newton shortcut used by the rejected Round 1 proof.

Why this repair iteration exists:

- A previous one-round run produced a Lean-accepted proof, but it imported ``Mathlib.Dynamics.Newton``.
- That proof used ``Polynomial.existsUnique_nilpotent_sub_and_aeval_eq_zero``.
- Mathlib's own ``LinearAlgebra/JordanChevalley.lean`` uses that Newton lemma internally.
- This repair round must avoid that shortcut and follow the lower-level blueprint route instead.

Prompt E: final verification. Purpose: run final verification without changing the proof.

The next task is final verification only.

Important:

Do not change the theorem statement.

Do not modify the proof unless a final verification command reveals an actual problem.

Do not rerun the proof loop.

The full prompt files, Numina config files, wrapper adapter, run logs, and final verification artifacts are included in the attached `agent_config_and_prompts.zip`.

4. Run Account and Iterations

I set up Lean 4 and Mathlib, inspected lower-level Mathlib tools for generalized eigenspaces, polynomial evaluation, nilpotence, semisimplicity, ideals, and CRT, then seeded the Lean file with the theorem statement and one temporary sorry. Archon was blocked by Claude authentication; the Archon-to-Codex patch was judged too large.

Iteration summary: the toy dry run tested the wrapper on `NuminaAdapterToy.lean`, where Codex replaced sorry with `trivial`. Round 1 on the real theorem completed technically, but I rejected it because it used `Mathlib.Dynamics.Newton` and a Newton lemma used internally by Mathlib's Jordan-Chevalley proof. Round 2 was the required steering/repair iteration: I restored the backup, added stricter bans, and Numina/Codex produced the final accepted proof. Final verification then ran Lean, placeholder searches, forbidden-name searches, and `#print axioms`.

The Round 2 adapter hit its 600-second timeout after Codex wrote a completion message; therefore, I relied on independent Lean and grep verification, not only the harness status.

5. Final Verification

```
lake env lean Formalization.lean
no output, Lean accepted the file.
```

```
rg -n "sorry|admit|axiom|constant|unsafe" Formalization.lean
no matches
```

I also searched `Formalization.lean` for forbidden or strict-forbidden names, including:

```
Mathlib.LinearAlgebra.JordanChevalley
exists_isNilpotent_isSemisimple
isNilpotent_isSemisimple_unique
exists_isNilpotent_isSemisimple_of_separable_of_dvd_pow
Dunford
Mathlib.Dynamics.Newton
existsUnique_nilpotent_sub_and_aeval_eq_zero
eq_zero_of_isNilpotent_isSemisimple
```

Result: no matches.

Exact axiom output:

```
'AI4MATH.jordan_chevalley_decomposition' depends on axioms: [propext,
Classical.choice, Quot.sound]
```

There is no `sorryAx`. The proof is not choice-free, since `Classical.choice` appears in the printed axiom dependencies. This dependence is expected for the present Jordan–Chevalley formalization: the theorem is stated over an algebraically closed field and finite-dimensional vector spaces, and the proof uses Mathlib tools for eigenvalue existence, generalized eigenspace decompositions, semisimplicity, and finite-dimensional decompositions, which typically rely on nonconstructive classical reasoning. Therefore, this Lean proof retains these standard classical dependencies as part of the final formalization, rather than presenting the result as a fully choice-free development.

6. Reflection

The agent did well at navigating Mathlib, repairing Lean errors, and turning the blueprint into a working proof using generalized eigenspaces, CRT-style polynomial construction, polynomial evaluation, nilpotence, semisimplicity, commutation, and uniqueness.

The hard part was avoiding proof routes that were too close to Mathlib’s built-in Jordan–Chevalley proof. Round 1 showed that a theorem can compile and still be unacceptable for an assignment boundary. I learned that autoformalization requires prompt design, harness adaptation, iteration, and audit: the final trust came from Lean kernel verification, explicit forbidden-name searches, and the exact `#print axioms` output.